Alireza Garmsiri

40260337

Evolutionary Computing

HW 2

INTRODUCTION

This project addresses the bin packing problem using a genetic algorithm (GA) to find an efficient way of packing items of different sizes into a minimum number of bins, each with a fixed capacity. The GA starts with a population of potential solutions, where each solution is represented as a chromosome that assigns items to specific bins. The algorithm then iterates through generations, using genetic operators such as selection (to choose the best solutions), crossover (to mix parent solutions and create new offspring), mutation (to introduce small changes for diversity), and elitism (to retain top-performing solutions). The fitness function drives the algorithm towards solutions that use fewer bins and minimize wasted space within bins.

CODE

1. Chromosome Class

The Chromosome class represents a potential solution to the bin packing problem, with each chromosome being a list of genes where each gene denotes the bin assigned to an item. Key methods and attributes include:

- **_generate_random_chromosome**: Generates a random initial chromosome by randomly assigning items to bins while respecting bin capacity constraints.
- **calculate_fitness**: Computes the fitness score for the chromosome based on the number of bins used and the wasted space in each bin. The fitness score is inversely proportional to the number of bins and the amount of unused space, rewarding more efficient solutions.

2. GAOperators Class

The GAOperators class includes methods for genetic operations:

- **tournament_selection**: Selects two parent chromosomes from the population based on a tournament-style competition, favoring higher fitness scores.
- **one_point_crossover**: Combines two parent chromosomes at a random crossover point to produce offspring, introducing diversity into the population.
- **swap_mutation**: Randomly swaps two genes within a chromosome to introduce small changes, helping prevent premature convergence to a suboptimal solution.
- **_validate**: Ensures that the generated chromosomes after crossover or mutation respect bin capacity constraints.

3. BinPackingGA Class

The BinPackingGA class manages the overall genetic algorithm process. Key attributes and methods include:

- **Run** : Executes the genetic algorithm over multiple generations, maintaining and evolving a population of chromosomes.
- **_create_new_population** : Creates a new population by applying selection, crossover, and mutation to the existing population.
- **_apply_elitism** : Preserves a percentage of the best solutions across generations, ensuring that the algorithm retains high-quality solutions.
- **plot_fitness_trends** : Visualizes the improvement in the best and average fitness scores over generations.
- **plot_histogram_of_bins** : Plots the initial and final distributions of bin usage to illustrate the GA's effectiveness in optimizing bin allocation.
- **plot_bins_like_image** : Provides a visual representation of the final bin configuration, showing the items and their sizes in each bin.

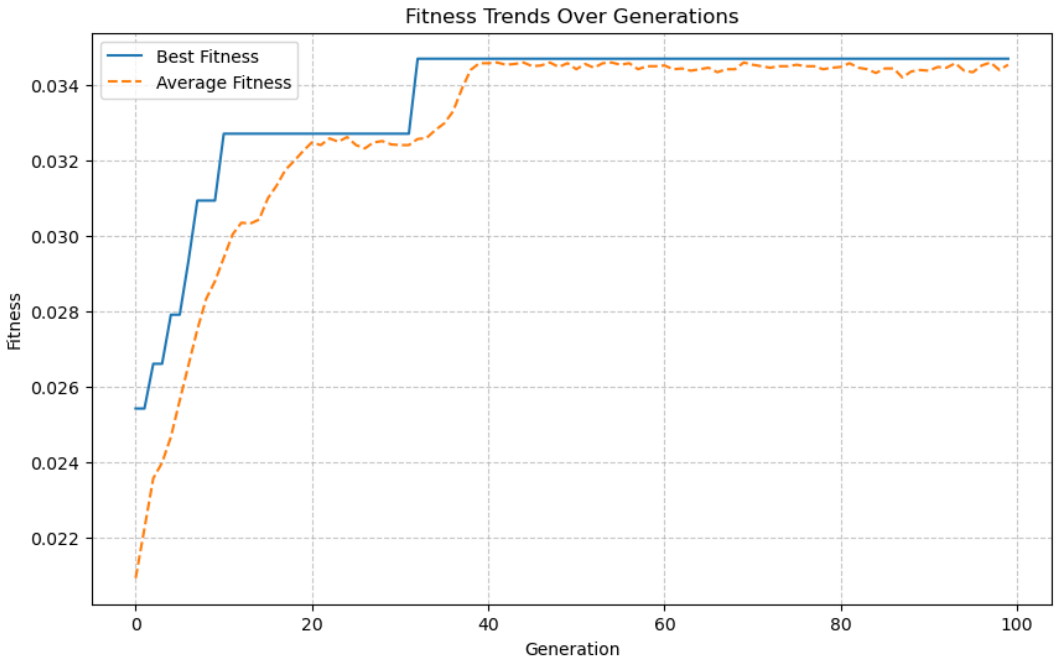
Methodology

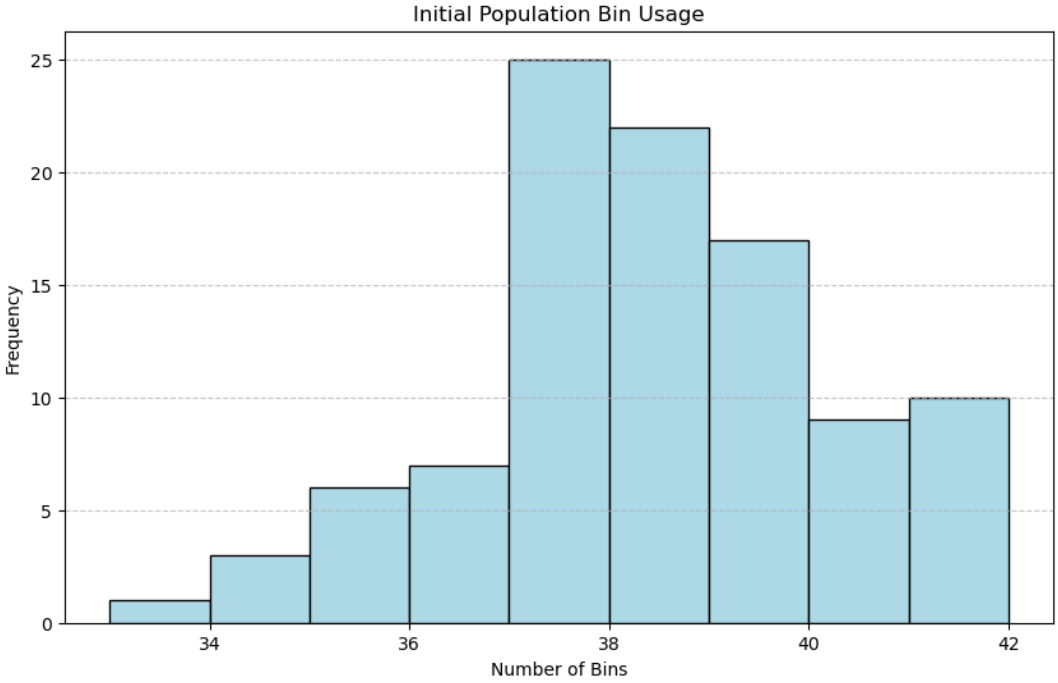
The genetic algorithm begins by creating an initial population of random chromosomes. Each generation involves:

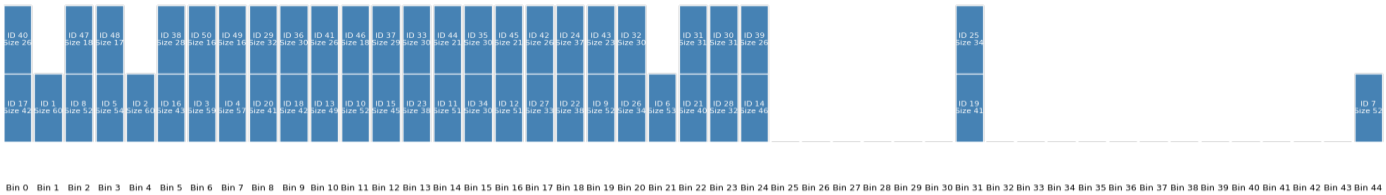
1. **Fitness Calculation**: Each chromosome's fitness score is calculated based on the number of bins used and the space wasted.
2. **Selection**: Tournament selection picks the best chromosomes as parents.
3. **Crossover and Mutation**: Crossover combines genes from two parents to create offspring, and mutation introduces random changes in some offspring to enhance diversity.
4. **Elitism**: A portion of the best solutions is preserved in the new generation to maintain progress towards optimality.
5. **Population Update**: The new population replaces the old one, and the process repeats.

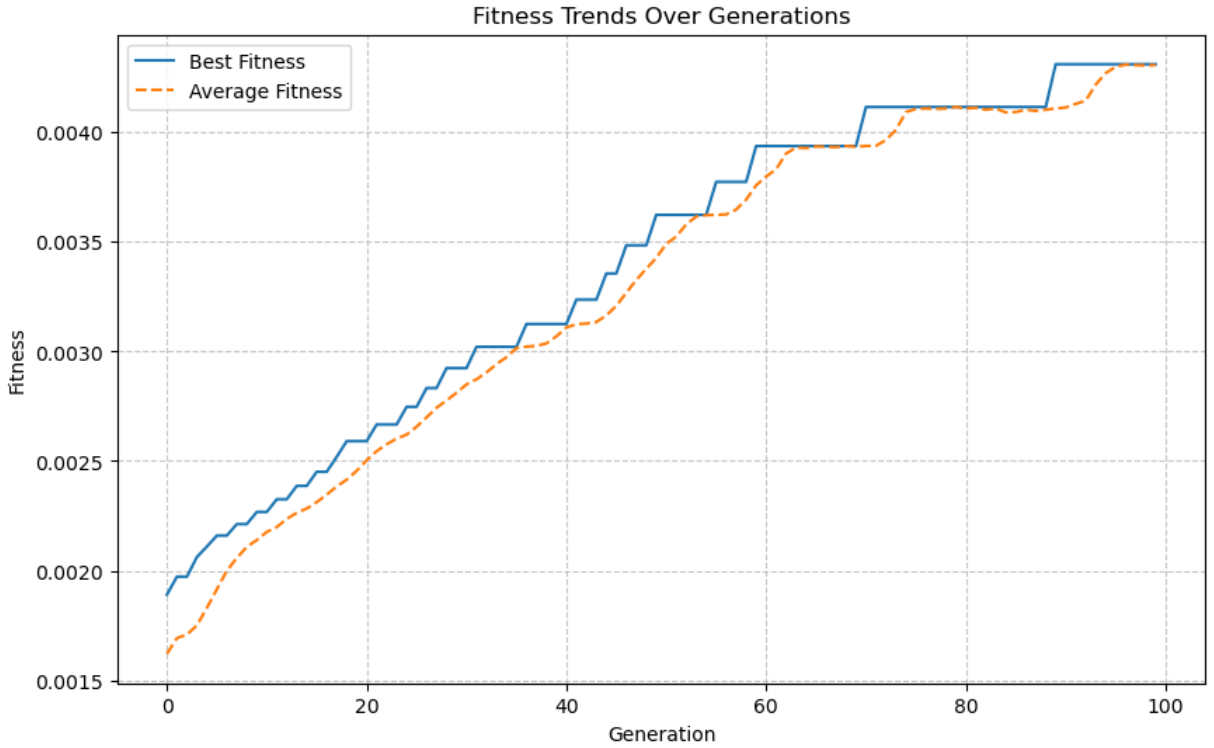
RESULT

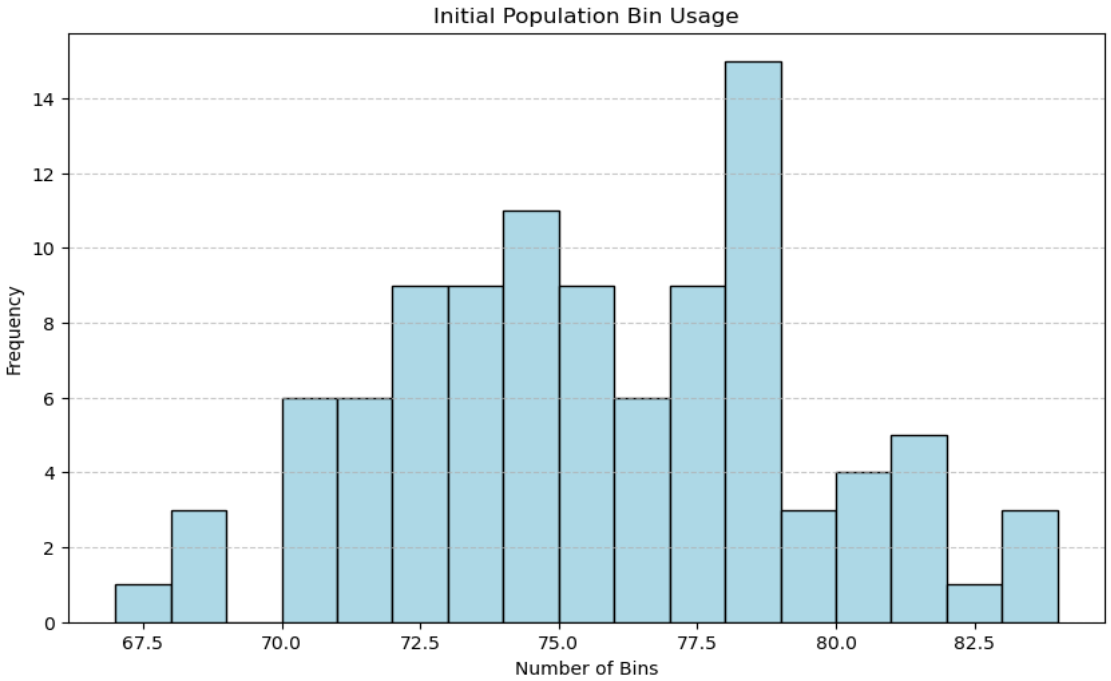
TXT 1

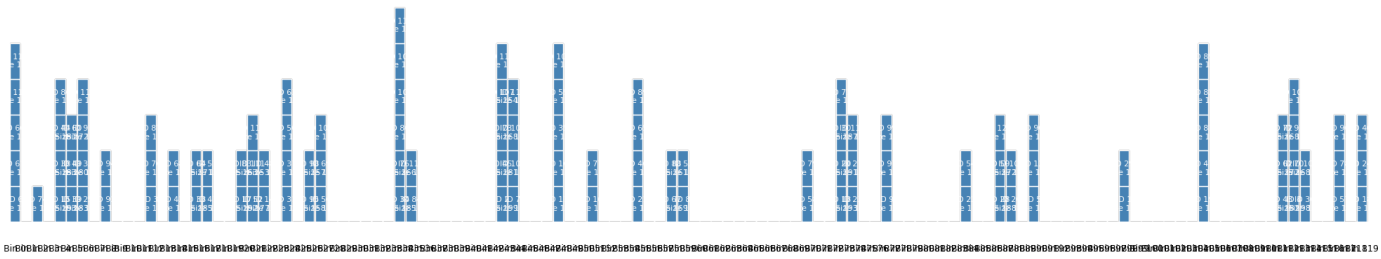

Fitness Trends Over Generations
Best Fitness
Average Fitness
Fitness
Generation
0.034
0.032
0.030
0.028
0.026
0.024
0.022
0
20
40
60
80
100


Initial Population Bin Usage
Frequency
Number of Bins
25
20
15
10
5
0
34
36
38
40
42




Fitness Trends Over Generations
Best Fitness
Average Fitness
Fitness
Generation
0.0040
0.0035
0.0030
0.0025
0.0020
0.0015
0
20
40
60
80
100


Initial Population Bin Usage
Frequency
Number of Bins
14
12
10
8
6
4
2
0
67.5
70.0
72.5
75.0
77.5
80.0
82.5



CONCLUSION

Demonstrates the power of genetic algorithms (GAs) in addressing complex optimization problems like bin packing. By simulating natural selection, the algorithm iteratively refines a population of solutions to minimize the number of bins required for storing items with varying sizes. The combination of selection, crossover, mutation, and elitism allows the GA to balance exploration and exploitation, ultimately converging towards a highly efficient configuration.

The fitness trend plot, initial and final bin usage histograms, and bin layout visualization effectively show the GA's optimization process and improvements over generations. The final solution is not only optimized in terms of bin usage but also reduces wasted space, making it a practical approach for real-world applications. Overall, this code showcases the adaptability and effectiveness of genetic algorithms in finding near-optimal solutions in complex, multi-variable scenarios.